

Following along class code

Santiago Del Moral

2026-09-16

Code will start here Important to write {r} to execute R code

```
x <- c(1, 2, 3)
length(x)
```

```
## [1] 3
```

```
typeof(x)
```

```
## [1] "double"
```

```
x
```

```
## [1] 1 2 3
```

something should've printed

```
i <- c(1L, 2L, 3L, NA_integer_)
typeof(i)
```

```
## [1] "integer"
```

```
i
```

```
## [1] 1 2 3 NA
```

We can have logical operators too && means AND || means OR TRUE means TRUE FALSE means FALSE NA means NA and will return NA in operators that need it

```
TRUE && TRUE
```

```
## [1] TRUE
```

```
TRUE && FALSE
```

```
## [1] FALSE
```

```
TRUE && NA
```

```
## [1] NA
```

```
TRUE || NA
```

```
## [1] TRUE
```

```
FALSE || NA
```

```
## [1] NA
```

There's also matrixes use `matrix()` y is the number of dimention use "drop=FALSE" to be able to have onedimentional matrixes instead of a vector.

```
h <- matrix(1:10, 1)
h
```

```
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
## [1,]    1    2    3    4    5    6    7    8    9    10
```

there are lists that can contain any data type data frames are two dimentional data structures a `$` can help you fetch

```
df <- data.frame(a=1:10, b=11:20, c=letters[1:10])
df
```

```
##      a  b c
## 1    1 11 a
## 2    2 12 b
## 3    3 13 c
## 4    4 14 d
## 5    5 15 e
## 6    6 16 f
## 7    7 17 g
## 8    8 18 h
## 9    9 19 i
## 10  10 20 j
```

```
df$a
```

```
## [1] 1 2 3 4 5 6 7 8 9 10
```

Functions

R is a functional programming language

```
add <-function(x,y) {
  x+y
}

add(1,2)
```

```
## [1] 3
```

more functions apparently you can type " " instead of function In R things are considered pass by value not pass by reference so x is considered immutable... which means unless you're doing <- then only the function's copy is being updated Original remains unchanged, unless assigned to a new variable

```
x <- 1:10
foo <- function(x){
  x[1] <- -99
  x
}

foo(x)
```

```
## [1] -99  2  3  4  5  6  7  8  9 10
```

```
#we'll see x is unchanged
x
```

```
## [1] 1 2 3 4 5 6 7 8 9 10
```

```
x<-foo(x)
x
```

```
## [1] -99  2  3  4  5  6  7  8  9 10
```

```
#now x is changed because it's been assigned
```

No need to keep track of mutations and changes because of immutability.

BREAK

Data visualization now Why do we look at data? because the summary statistics can be misleading sometimes can check intuitively for outliers find patterns visually

Pie charts are not the best usually, plain bar charts are better QQ plot, a empirical in the y axis and the theoretical in the x axis (usually normal dist)

Looking at multiple variables, how to check relationships? scatterplot jitter to avoid overplotting in a scatter plot can work to break up things

density plots 1D or 2D are cool

Plotting: we'll be using library(ggplot2) for now

```
library(ggplot2)
ggplot(penguins,
       aes(x=bill_len,y=body_mass))+geom_point()
```

```
## Warning: Removed 2 rows containing missing values or values outside the scale range
## ('geom_point()').
```

